COSC204
Computer Systems

Arrays, Malloc, and Free

Overview

- Last Lecture
  - Memory, Variables, and Pointers
- This Lecture
  - Arrays, Malloc, and Free
  - Source
    - K&R:
      - Brian Kernighan and Dennis Ritchie (1988), The C Programming Language. 2nd Edition. Prentice Hall. ISBN:0-13-110362-8
- Next Lecture
  - Structs and Unions

Arrays

- C arrays are a *little bit* like Java arrays
- To declare an array we must know the size

- Give me an array large enough to hold 10 unsigned 32-bit integers

```
uint32_t array[10];
```

- Give a 2-dimensional array (a table, or matrix) that has 5 rows and 6 columns

```
float matrix[5][6];
```

Array Methods

- Arrays in C do not have methods
  - You cannot ask an array how large it is
  - You cannot simply append to an array
  - You cannot copy an array with a=b
  - Arrays are not bounds checked
    - You ***can*** write past the end of the array
- To know the number of elements in an array you must either keep track of it yourself, or put a *sentinel value* on the end and count
- A sentinel value is an element that means “end”

Sentinel Values

- An array of 6 characters called `thing`, containing the characters ‘H’, ‘e’, ‘l’, ‘l’, and ‘o’
- Add a non-printable character to mark the end (‘\0’)

```
char thing[6];
```
← 6 characters (numbered from 0 to 5)

```
thing[0] = 'H';
thing[1] = 'e';
thing[2] = 'l';
thing[3] = 'l';
thing[4] = 'o';
thing[5] = '\0';
```
← “Thing 1” reads better than “Things 1”, so we prefer array names in the singular

← Sentinel value of ‘\0’ on the end

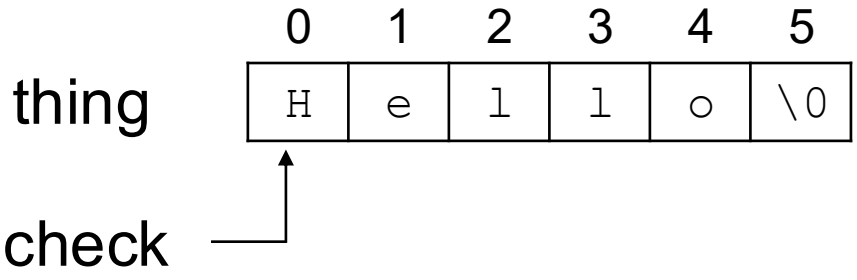
Length of an array

- If there is a sentinel on the end then we can find the number of things in the array by counting them

```
const char *check = thing;

while (*check != '\0')
  check++;

length = check - thing;
```



C Can't Pass Arrays By Value

- In C you *cannot* copy an array with `a=b`
  - You can't pass an array by value
  - The name of the array is short-hand for `&thing[0]`
- So when we go

```
get_length(my_array);
```

- What we really mean is

```
get_length(&my_array[0]);
```

- So what we receive in `get_length` is a pointer

```
get_length(char *element)
```

`get_length()` or `strlen()`

- Strings in C are arrays of characters
- `strlen()` returns the length of a string
- We've just derived C strings from first principles!

```
size_t strlen(const char *of)
{
const char *check = of;

while (*check != '\0')
   check++;

return check - of;
}
```

What About “Quoted” Strings

- Quoted strings, or *string literals*, look like this

```
"Hello world"
```

- You are not allowed to change them, so they are `const`

```
const char *hello_world = "Hello world";
```

- Because they’re like arrays, you can use them almost anywhere you would use an array of characters

```
printf("%zu\n", strlen("Hello world"));
```

Initialising An Array

- We've already hinted on how to initialize an array
- In this case we only give values to 3 of the 5 elements, the others are initialised to 0
- `double balance[5] = {1000.0, 2.0, 3.4};`
- In this case we say "work it out the size for me", but it does have a size (in this case 3)
- `double balance[] = {1000.0, 2.0, 3.4};`

Memory

- When we declare an array what we are saying is "choose somewhere in memory to put `this` number of `that` and call it `thing`"

```
that thing[this];
```

```
uint8_t byte_array[1024];
```

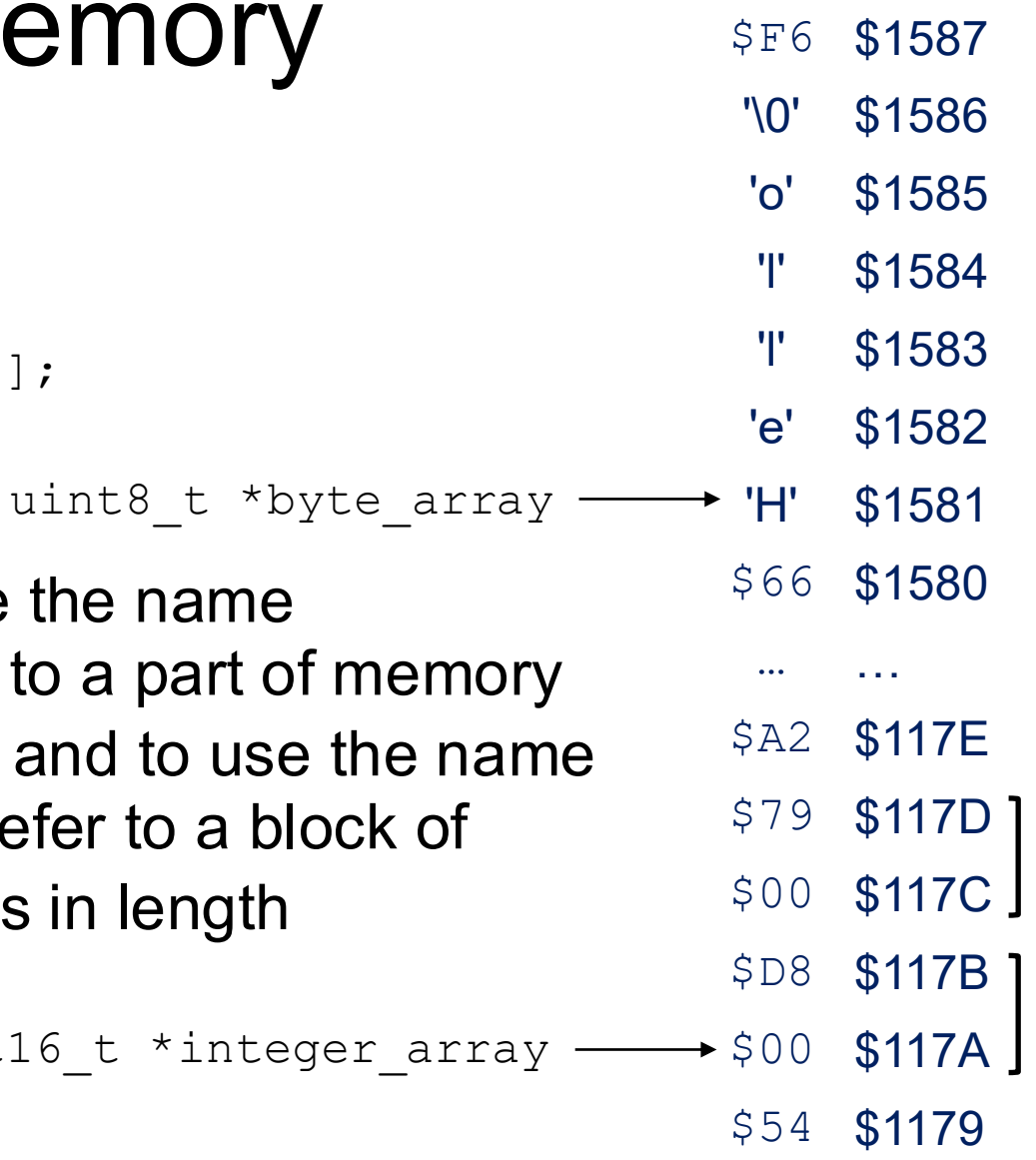
- Remember that large array of bytes we call memory?
  - The compiler will choose somewhere in it to put the array
  - The name of the array (`thing`, `byte_array`) is the location, the place where the compiler put the array

Memory

- For example,

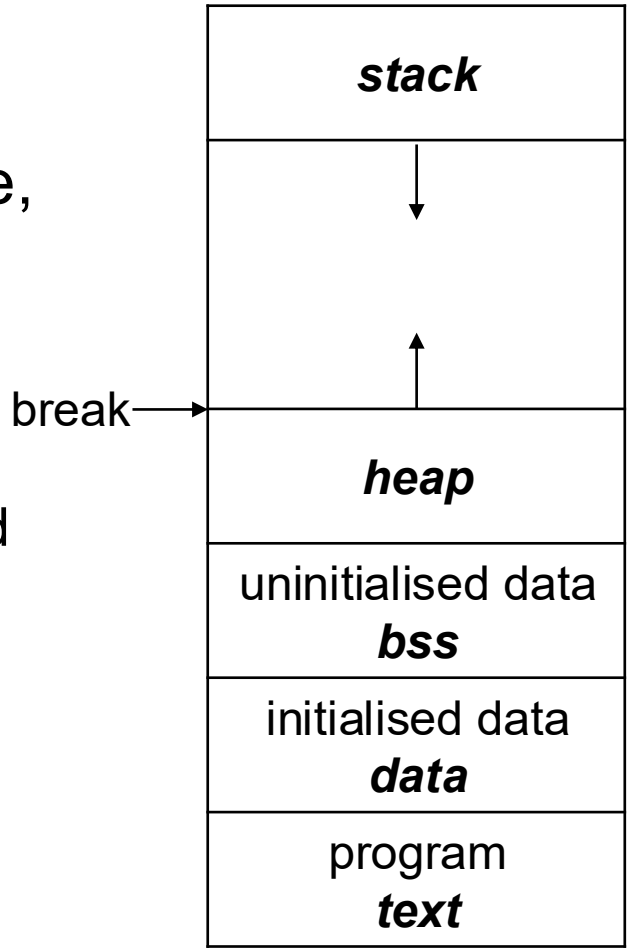
```
uint8_t byte_array[6];
uint16_t integer_array[2];
```

- Tells the compiler use the name `byte_array` to refer to a part of memory that takes up 6 bytes, and to use the name `integer_array` to refer to a block of memory that is 4 bytes in length



Segments

- What if we don't know, at compile time, how big an array needs to be?
- The compiler knows:
  - Where, in memory, your program is stored
  - How much global data you use
  - Where the stack is
- Everything else is unused: The *heap*
  - The compiler knows where this starts



malloc()

- What if we don't know, at compile time, how big an array needs to be?
  - We can get space from the heap
- *Find 16 consecutive bytes of memory somewhere in the heap and tell me where it is (so that I can assign its address to a variable)*

```
uint8_t *space = malloc(16);
```

- Now `space` is just like an array of 16 bytes because, remember, `space` is the short way of saying `&space[0]`

malloc()

- C Gotcha:

- *Allocate space to store 16 unsigned 64-bit integers*

```
uint64_t *block = malloc(sizeof(uint64_t) * 16);
uint64_t *another = malloc(sizeof(*another) * 16);
```

- Because `malloc()` allocates in units of bytes we need to know how many bytes each `uint64_t` takes and multiply by the number we want

free()

- Unlike Java, in C, ***when you have finished using an allocated block you must free that block***

```
uint64_t *block = malloc(sizeof(*block) * 16);
/* do something */
free(block);
```

- Never free memory that you didn't allocate
  - You can't just free() an arbitrary pointer
- Never free the same memory twice
  - This will "break" the dynamic memory management routines
- You ***must*** call free before you lose the pointer to the block of memory

Bugs To Look Out For

- Malloc'd block is too small
  - Buffer-overrun
- Didn't free a block
  - Memory leak
- Malloc and free small units a large number of times
  - Memory fragmentation
- Free the same block more than once
  - Corrupt the memory management routines
- Using memory after it has been freed
  - It may have been re-used by malloc

Homework

- Define the following terms:
  - Sentinel value
  - String literal
  - The heap

- Write some more of the string routines yourself (in C)
  - char *strcpy(char *destination, const char *source);
  - char *strdup(const char *s);
    - This requires you to call malloc()